# Test Time Compute Main Presentation Script

## Slide 1 Title page

Today I will present a paper about using more computation when a language model answers a question.

## Slide 2 Can Language Models Benefit from More Thinking

Pass

## Slide 3 How Do We Work through a Math Problem

Pass

## Slide 4 Two Directions for Using Extra Computation

Pass

## Slide 5 PRM Guided Search from Steps to an Answer

Pass

## Slide 6 Who Can Score the Reasoning

Pass

## Slide 7 How Do We Select the Final Answer

Pass

## Slide 8 Which Candidates Should We Develop

The third question is how to generate and keep the candidates. The three diagrams show three choices.

Best-of-N independently generates complete solutions and selects afterward.

Beam search brings the verifier into the generation process. At each depth, it scores the current prefixes, keeps the highest-scoring ones, and expands them again.

Lookahead temporarily continues for up to k more steps, then uses the PRM score at that endpoint to evaluate the current prefix. The temporary continuation helps choose where to search next.

This also costs extra computation. The comparison charges those additional steps to the budget. If k is zero, lookahead reduces to ordinary beam search. We now have a trained scorer, a search procedure, and a final selection rule. We can compare how well they work.

## Slide 9 Beam Search Helps Most at Low Budgets

The experiments use PaLM 2-S star on the MATH benchmark, with 12,000 training questions and 500 test questions. This graph compares search methods.

The horizontal axis is the generation budget. The vertical axis is accuracy. To compare methods fairly, first look at the same horizontal position. At small budgets, beam search is stronger than weighted best-of-N. As the budget grows, this advantage shrinks, and best-of-N can catch up or pass it.

Lookahead usually gives less benefit at the same estimated cost. It spends some of its budget on temporary continuations.

## Slide 10 Difficulty Changes Which Search Strategy Wins

The authors divide questions into five difficulty groups relative to the base model. Group one is easiest and group five is hardest.

This graph needs a careful reading. Each group contains four positions, for budgets of 4, 16, 64, and 256. At each position, three colored bars overlap. Every bar starts at zero. We read the top of each color, rather than add the visible colored pieces.

Blue is beam search, orange is weighted best-of-N, and green is majority voting. For example, in the easiest group at budget four, beam search is highest. As the budget grows, the independent methods catch up. The paper finds signs of over-optimization by beam search in the easy groups.

In groups three and four, beam search is consistently stronger than best-of-N. In group five, all the methods make little progress.

So failures are not limited to the hardest questions. Extra search can hurt easy questions, help medium difficulty questions, and still fail on the hardest ones. This gives us a reason to choose settings using both difficulty and budget.

## Slide 11 Less Compute Can Reach Similar Accuracy

Choosing the right search setting helps us reach similar accuracy with less compute.

Look at the green line, PRM best-of-N weighted, at sixty-four generations. Now look at the blue Oracle line at sixteen generations. The accuracy is almost the same. The orange Predicted line at sixteen also comes close to the green line at sixty-four.

So choosing the search setting by difficulty can reduce the compute needed for a similar result. This figure does not include the compute used to estimate difficulty. Next, we let the model revise its earlier answers.

## Slide 12 Revising Answers and Selecting the Output

## Slide 13 Revision Quality and Final Answer Accuracy

Let's start with the left figure. For one question, the model runs four separate chains. Each chain has sixty-four complete answers. At round ten, we check the tenth answer in each chain. We average correctness over the four chains, then over all 500 test questions.

The four chains help us estimate average accuracy. This figure uses no majority vote or ORM selection, either within a chain or across chains. Most improvement happens in early rounds.

Now look at the right figure. Suppose N is eight. Sequential generation makes eight complete answers in one chain, using earlier answers to write the next one. Parallel generation makes eight independent answers. Both use the same revision model.

For sequential generation, we choose from the eight answers in that single chain. Majority voting counts how often each final answer appears. Weighted best-of-N sums ORM scores for matching final answers. We apply the same two rules to the eight independent answers in the parallel case.

Two generation styles and two selection rules give four curves. We check the selected answer for each question and measure accuracy across all 500 questions. Compare sequential and parallel with the same selector: sequential is slightly better. The next slide combines several revision chains and studies how many chains to use and how long to make them.

**Conclusions shown on the slide**

Most improvement happens in early rounds.

Sequential is slightly better with either selector.

## Slide 14 The Best Mix Depends on Difficulty and Budget

Now we combine independent starts with sequential revisions. The example below uses 128 answers: eight chains, with sixteen complete answers in each chain.

The ORM first makes a weighted choice inside each chain. It sums scores for matching final answers, then keeps the highest-scoring solution for the winning answer. We now have eight representatives. A second weighted choice across these eight gives one final answer.

Both figures use this same selection process. In the left figure, each line keeps the total budget fixed. The horizontal axis is chain length divided by chain count. Our example has ratio two. At high budgets, a mix can beat both extremes.

The right figure fixes the total budget at 128. Each group is a difficulty level, and the colors show different ratios. Easy questions favor longer chains. Harder questions often need more independent starts.

**Conclusions shown on the slide**

At high budgets, a mix can beat both extremes.

Easy questions favor longer chains; harder ones often need more starts.

## Slide 15 Adaptive Revisions Use the Generation Budget More Efficiently

This graph applies that adaptive choice. Colors distinguish oracle difficulty, predicted difficulty, and pure parallel generation. For solid lines, the ORM selects within each chain and then across chains. For dashed lines, all answers from all chains vote together.

Compare methods with the same line style. At larger budgets, pure parallel sampling begins to plateau, while the adaptive mixtures continue improving. In the reported comparison, sixty-four candidates can beat parallel best-of-N with 256 candidates.

Both difficulty definitions help, although estimating difficulty has an extra cost excluded from the plotted budget.

We have now seen gains from search and from revisions. Can these gains make a small model competitive with a larger one?

**Conclusions shown on the slide**

Adaptive revisions keep improving after parallel sampling begins to plateau.

64 candidates can beat parallel best-of-N with 256 candidates.

## Slide 16 Pretraining and Inference at Equal Total FLOPs

To answer that, we must compare total computation. Training a larger model costs more, and using it to generate answers also costs more.

Let P be the small model's parameter count. Its pretraining cost is approximately six times P times the number of training tokens. Its inference cost is approximately two times P times the total inference tokens. We call these costs X and Y.

If parameters grow by a factor M, both costs grow by M. The total is M times the sum of X and Y.

The small model still spends X on training. We choose its inference multiplier a so that X plus aY equals the larger model's total cost. This equation sets an equal budget.

## Slide 17 Using R to Compare Total Compute

R is the ratio of inference tokens to pretraining tokens. We use this ratio for the cost comparison. R is different from a, the small model's inference budget multiplier.

The small-model curve tells us the accuracy at each inference budget. R tells us which budget has the same total cost as the larger model. This total includes pretraining and inference.

Start with the equal-cost equation from the previous slide. Solve for a, replace X and Y with their cost formulas, and cancel the parameter count. Six divided by two gives three. The training-to-inference token ratio is one divided by R. This gives the final formula.

For one fixed R, only the matched value of a gives equal total cost. If we give the small model more compute than this, its total cost becomes higher. We can now compare accuracy at the matched positions in the next figure.

## Slide 18 Extra Inference Can Beat a Larger Model in Some Settings

The curves show the small model with adaptive extra inference. The first figure uses revisions and the second uses PRM search. Colors represent difficulty.

The horizontal axis is proportional to inference compute. Each vertical line is a matched-total-cost point for one R value. We compare the same color at that line. If the curve is above the star, extra inference on the small model is more accurate. If the star is above the curve, the larger model is more accurate.

The stars show greedy accuracy from a model with about fourteen times more parameters.

The same large-model star appears at several horizontal positions with the same height. Its accuracy has stayed the same. The horizontal position changes because expected use changes the small model's affordable inference budget.

Easy and medium questions often favor extra inference, especially with lower expected use. Very hard questions often favor the larger model. Higher expected use also leaves less extra inference for the small model.

These are conditional findings. The comparison uses oracle difficulty groups and fixed pretraining token counts. These findings lead to the final conclusions.

## Slide 19 Conclusions

22:15 to 24:00

The experiments show that a model can improve its answers with more compute at inference time. The best use of that compute changes with the question and the budget.

For PRM search, beam search works better on harder questions and at low budgets. On easier questions or at high budgets, best-of-N is often better.

For revisions, easy questions favor longer chains. Harder questions often need a mix: revise earlier answers, but also try several independent starts.

Choosing the setting by difficulty and budget gives concrete savings. Adaptive search at sixteen generations nearly matches best-of-N at sixty-four. Adaptive revisions with sixty-four generations can beat parallel best-of-N with two hundred and fifty-six. These budgets exclude the compute used to estimate difficulty.

Finally, at equal total compute for pretraining and inference, the smaller model with extra inference can beat a model with about fourteen times more parameters. This works on questions within its capabilities and often with a low inference workload. For very hard questions or a high inference workload, extra pretraining is often more effective.

## Slide 20 Questions

Thank you for listening. I am happy to take questions.